SoloSphere

FULL STACK PROJECT REPORT

A Community-Driven Safety Platform for Solo Travelers

Tech Stack:	React 19 · Node.js · Express · TypeScript · MongoDB
Architecture:	MVC + Three-Layer · REST API · JWT Auth
Deployment:	Vercel (Frontend & Serverless Backend)
Repository:	github.com/connectwithvanshika/Solosphere-FS

April 2026 · Academic Project Report

01 Problem Statement & Solution Approach

Problem Statement

Solo travel is growing rapidly worldwide, yet the digital ecosystem for solo travelers remains fragmented and, crucially, unsafe. Women and first-time travelers face specific challenges that existing platforms fail to address: accommodation reviews are unverified and often commercially biased; emergency resources are scattered across dozens of sites; community-driven local guidance is absent; and finding trustworthy travel companions with compatible itineraries requires extensive manual research across multiple social networks.

Pain Point	Impact
Unverified Information	Reviews on generic platforms lack safety context — ratings reflect hospitality, not personal s
Emergency Preparedness Gap	No single platform aggregates local emergency numbers, safe zones, and real-time SOS cap
Companion Discovery	Finding compatible travel companions requires manual cross-platform searching with no safe
Community Fragmentation	Safety tips, local insights, and verified safe places are scattered across Reddit, Facebook gr

Solution Approach

SoloSphere addresses these challenges through a unified, community-first web platform built on a modern full-stack architecture. The solution consolidates verified safe places, community safety reviews, companion matching, travel tips, and emergency SOS functionality into a single, authenticated, role-governed application.

Core Solution Pillars

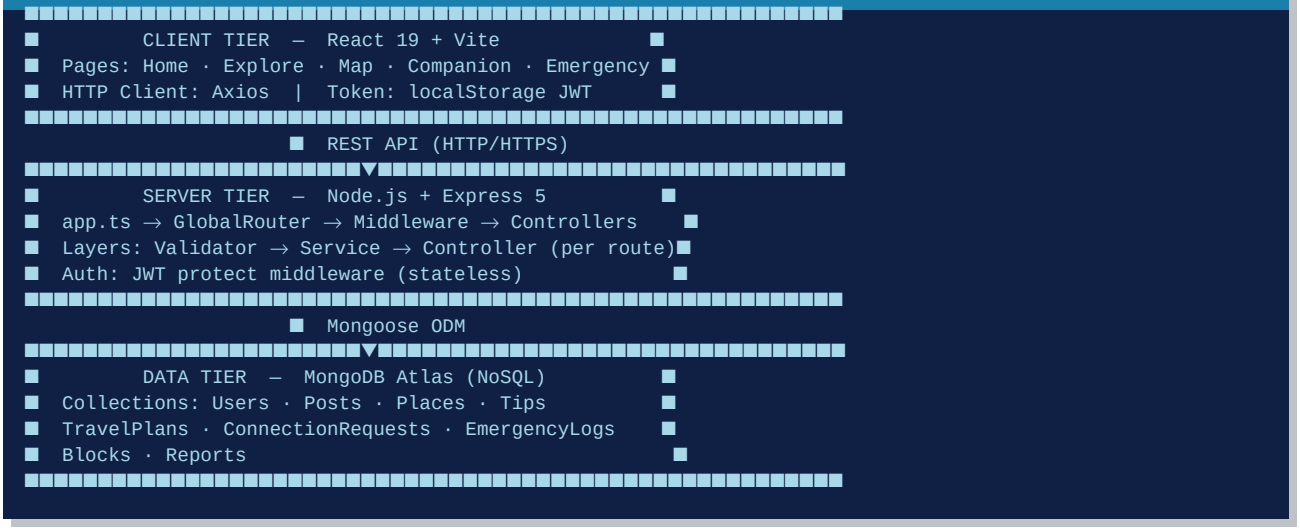
- Verified Safe Places — Hostels, cafés, campsites with community safety ratings
- Safety-First Community Reviews — User-generated content tied to real accounts
- Companion Matching — Filter by city, date, gender preference
- Emergency SOS — One-tap GPS-tagged alert logging with admin visibility
- 60+ Curated Travel Tips — Safety, transport, wellness, helplines
- Safety Controls — Block, report, and community moderation layer
- JWT Authentication — Stateless, scalable, role-based access control

02 System Design Optimization

Three-Tier Architecture

SoloSphere employs a strict client–server–database three-tier model. The React frontend communicates exclusively through a REST API surface, never touching the database directly. The Express backend mediates all data access through Mongoose ODM. This separation enables independent horizontal scaling of each tier.

System Architecture Overview



Performance Optimisations Applied

Parallel DB Queries	Promise.all([find(), countDocuments()]) halves round-trip latency on paginated endpoints.
Lean Queries	.lean().exec() returns plain JS objects instead of Mongoose Documents — ~2× faster for read-only endpoints.
Pagination on All Collections	Every list endpoint exposes page + limit, preventing unbounded memory usage.
Stateless JWT Auth	No server-side sessions; any instance handles any request — prerequisite for horizontal scaling.
Field Projection	Only required fields fetched from MongoDB; password field excluded from all response projections.
MongoDB Indexing Strategy	Compound indexes on (city, rating), (userId), (createdAt) on all hot collections.

Scalability Roadmap

Phase	Enhancement	Benefit
Phase 1	Redis cache for tips & popular places	Sub-10ms read latency
Phase 1	Cloudinary CDN for image storage	Offload static bandwidth
Phase 2	Microservice: Auth Service	Independent scaling & isolation
Phase 2	Notification Service (email/SMS)	Decoupled async processing
Phase 3	WebSocket SOS live alerts	Real-time safety broadcasts
Phase 3	ML companion matching engine	Preference-aware recommendations

03 Object-Oriented Programming Concepts

Encapsulation

Each route file wraps its Validator, Service, and Controller classes with clearly scoped private members. The PostController holds a private postService reference; external code never touches service internals. Mongoose schemas encapsulate schema shape, validation hooks, and default values behind a clean Document interface.

Code Evidence
`PostController.postService — private field`
`Mongoose Model hides schema internals`
`AuthController exposes only registerUser() / loginUser()`
`EmergencyService encapsulates GPS log logic`

Abstraction

TypeScript interfaces (ICreatePostPayload, IValidatedPostSearchQuery, ITravelPlanPayload) define contracts without exposing implementation. Consumers work against the interface, not concrete classes. The protect middleware abstracts the entire JWT verification flow into a single Express handler signature.

Code Evidence
`ICreatePostPayload — input contract`
`ITipsPaginatedResponse — output contract`
`protect() — abstracts token extraction + DB lookup`
`UserRepository — abstracts Mongoose from auth logic`

Polymorphism

Strategy objects returned by buildSortObject() and buildSearchFilters() are polymorphically interchangeable objects with the same shape but different runtime values. The error-delegation pattern lets any Error subclass flow through next(error) and be handled uniformly by the global error middleware.

Code Evidence
`Sort strategies: rating | reviews | recent`
`Filter strategies: city | tags | guestCount | nightSafety`
`Validator asserts return polymorphic typed payloads`
`Error types handled uniformly via next()`

Inheritance

Mongoose Schema inheritance through discriminators and shared base schemas. TypeScript interface extension — IValidatedTipQuery extends common pagination fields. Express Router composition via createXxxRouter() factory functions that all follow the same structural contract.

Code Evidence
`Mongoose Document — base class for all model instances`
`IValidatedTipQuery extends IPaginationBase`
`All route factories follow identical initialization protocol`
`Error classes extend base Error`

04 Design Patterns Implemented

Factory Pattern

Every route module exposes a createXxxRouter() factory function that instantiates the Service, injects it into the Controller, registers all sub-routes, and returns the populated Router. Callers receive a configured router without knowing how it was built.

Implementation — Factory Pattern

```
createPostRouter()
  postService = new PostService()
  postController = new PostController(postService)
  router.post('/', postController.createPost)
  return router
```

Dependency Injection

Controllers receive Service instances through constructor injection rather than creating them internally. This decouples the HTTP layer from business logic and makes unit testing trivial — a MockService can be injected in tests.

Implementation — Dependency Injection

```
class EmergencyController {
  constructor(private service: EmergencyService) {}
  logEmergency = async (req, res, next) => {
    const log = await this.service.logEmergencyActivation(...);
  }
}
```

Strategy Pattern

PlacesService.buildSortObject() returns one of three sort strategies (rating / reviews / recent) selected at runtime from query parameters. PostService.buildSearchFilters() composes up to four orthogonal filter strategies based on user input.

Implementation — Strategy Pattern

```
const sortMap = {
  rating:  { rating: -1 },
  reviews: { reviews: -1 },
  recent:  { createdAt: -1 },
};
return sortMap[sortField];  // strategy selected at runtime
```

Repository Pattern

UserRepository in protect.ts abstracts all Mongoose calls behind findUserById() and findUserByEmail(). Switching from MongoDB to another database only requires changing this single class.

Implementation — Repository Pattern

```
class UserRepository {
  async findUserById(id: string) {
    return User.findById(id);
  }
  async findUserByEmail(email: string) {
    return User.findOne({ email });
  }
}
```

Middleware / Chain of Responsibility

The Express middleware stack forms a Chain of Responsibility: CORS → BodyParser → Auth → Route Handler → Error Handler. Each handler either processes the request and passes it on or terminates the chain by sending a response.

Implementation — Middleware / Chain of Responsibility

```
app.use(cors())           // Link 1
app.use(express.json())   // Link 2
app.use(protect)          // Link 3 — auth
router.post('/', handler) // Link 4 — business
app.use(errorMiddleware)  // Link 5 — catch-all
```

Singleton Pattern

authController is exported as a singleton instance (export default new AuthController()). The same object is shared across all imports, avoiding duplicate state and saving memory for stateless handler classes.

Implementation — Singleton Pattern

```
// authController.ts
class AuthController { ... }
export default new AuthController();

// authRoutes.ts
import authController from '../controllers/authController';
```

05 SOLID Principles

S — Single Responsibility Principle

Every class has exactly one reason to change. PostValidator validates post payloads and nothing else. PostService executes business logic and never handles HTTP. PostController maps HTTP requests to service calls and never touches the database. UserRepository handles user queries only.

Component	SRP Evidence
PostValidator	Input validation only
PostService	Business logic only
PostController	HTTP request/response only
UserRepository	Database access only

O — Open/Closed Principle

The strategy maps in buildSortObject() and buildSearchFilters() can be extended with new entries without modifying existing code. Adding a 'safety-score' sort requires adding one key to sortMap — no existing code path changes. New route modules are registered by adding one line to GlobalRouter without touching existing routes.

Component	SRP Evidence
sortMap object	Add new sort key = zero change to callers
buildSearchFilters()	New filter = new if-block, existing filters untouched
GlobalRouter.registerRoutes()	New route = append line only

L — Liskov Substitution Principle

TypeScript interface contracts ensure every validator payload type is a subtype of its declared interface — any code expecting ICreatePostPayload works correctly with any value that satisfies that interface. Mongoose Document instances are liskov-safe substitutes for the plain JS objects described by their interface definitions.

Component	SRP Evidence
ICreatePostPayload	Subtypes satisfy full contract
Mongoose Document	Substitutable for plain schema objects
asserts payload is T	TypeScript enforces LSP at type level

I — Interface Segregation Principle

Interfaces are narrow and role-specific. ICreatePostPayload differs from IUpdatePostPayload; IValidatedPostSearchQuery differs from IValidatedTipQuery. Controllers receive only the service interface they need rather than a fat monolithic service.

Component	SRP Evidence
ICreatePostPayload vs IUpdatePostPayload	Different shapes for different operations
IValidatedPostSearchQuery	Search-only; no mutation fields
EmergencyController	Receives EmergencyService only, not full app service

D — Dependency Inversion Principle

High-level controllers depend on the Service abstraction, not on concrete Mongoose calls. Services depend on the UserRepository interface, not on User.findById() directly. The entire authentication flow is injected into the middleware via AuthenticationService constructor injection, inverting the dependency from concrete to abstract.

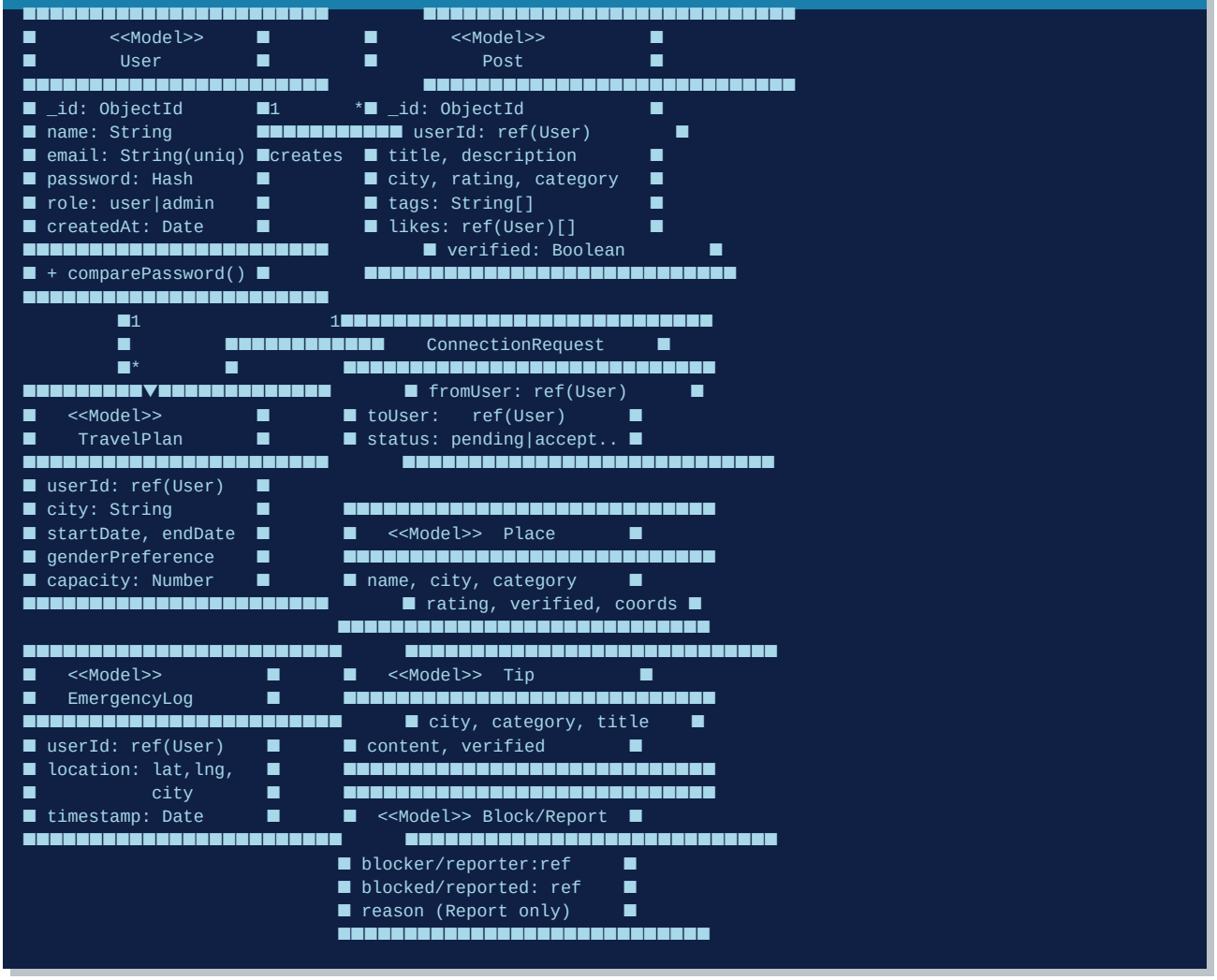
Component	SRP Evidence
EmergencyController	Depends on EmergencyService interface
AuthenticationService	Depends on UserRepository interface
protect middleware	Injected auth service, not hard-coded User.findOne()

06 UML Diagrams

6a — Class Diagram

The class diagram captures the nine core Mongoose model classes and their relationships. User is the central entity, referenced by every other collection. Posts carry likes as an array of User refs. Blocks and Reports form a safety graph between pairs of users.

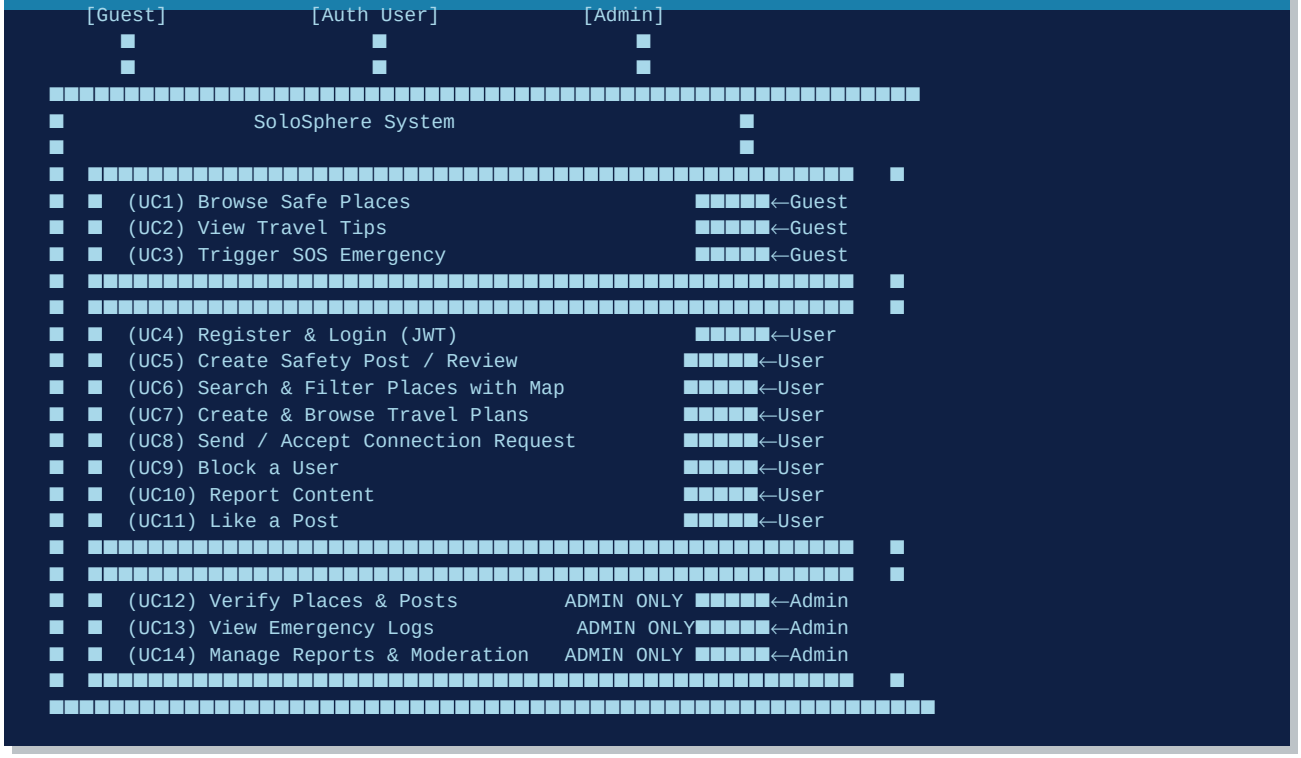
Class Diagram — Mongoose Models



6b — Use Case Diagram

Three actor types interact with the system: Guest (unauthenticated), Authenticated User, and Admin. The diagram shows the full scope of platform capabilities accessible to each actor.

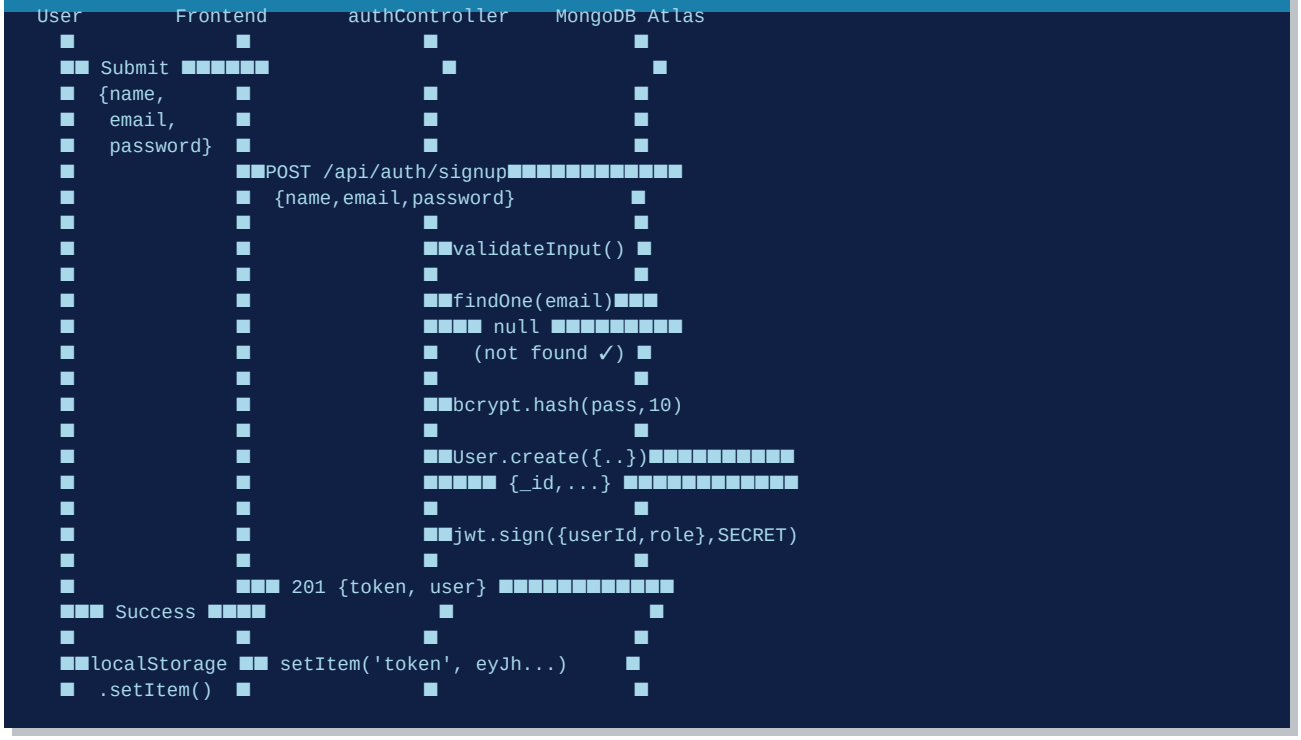
Use Case Diagram



6c — Sequence Diagram: User Registration & JWT Generation

The registration flow demonstrates the complete lifecycle from form submission through input validation, password hashing, database insertion, JWT signing, and token storage on the client.

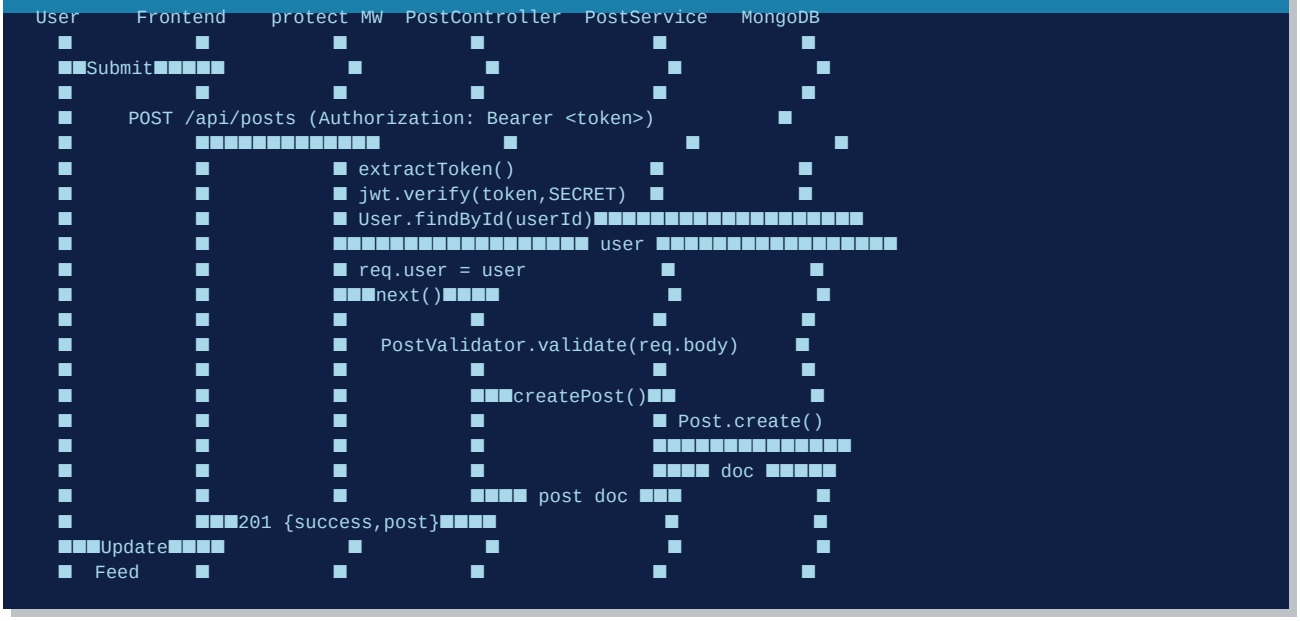
Sequence Diagram — Registration Flow



6d — Sequence Diagram: Authenticated Post Creation

This sequence illustrates the protected post creation flow, highlighting the JWT verification middleware as a cross-cutting concern that intercepts every mutating request before it reaches the business layer.

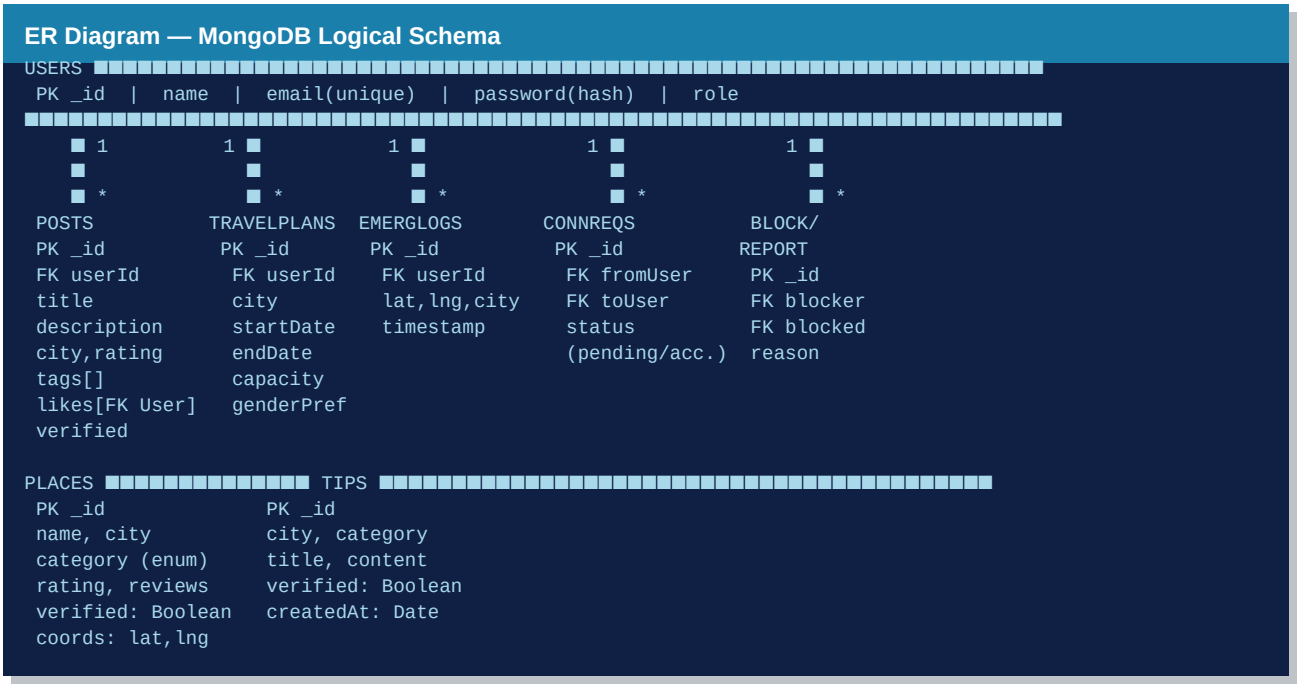
Sequence Diagram — Post Creation



6e — Entity-Relationship (ER) Diagram

MongoDB is document-oriented (schemaless at the engine level), but Mongoose schemas impose a relational-like structure via ObjectId references. The ER diagram below captures the logical entity relationships.

ER Diagram — MongoDB Logical Schema



07 Test Cases & Results

Authentication Module — Test Cases

TC-ID	Test Case Description	Input	Expected Output	Result
TC-A01	Valid user registration	name, email, password	201 + JWT token	PASS
TC-A02	Duplicate email registration	Existing email	400 — email already exists	PASS
TC-A03	Login with correct credentials	email + password	200 + JWT token	PASS
TC-A04	Login with wrong password	email + wrong pass	401 Unauthorized	PASS
TC-A05	Missing required field (name)	{email, password} only	400 Bad Request	PASS
TC-A06	Access protected route without token	No Authorization header	401 Unauthorized	PASS
TC-A07	Access protected route with expired JWT	Expired token	401 Token expired	PASS
TC-A08	Access protected route with valid JWT	Valid Bearer token	200 + data	PASS

Posts Module — Test Cases

TC-ID	Test Case Description	Auth	Expected	Result
TC-P01	Create post with all valid fields	Yes	201 + post doc	PASS
TC-P02	Create post with missing title	Yes	400 — title required	PASS
TC-P03	Create post with rating > 5	Yes	400 — rating 0-5 only	PASS
TC-P04	Search posts by city filter	No	200 + filtered list	PASS
TC-P05	Paginated post search (page=2)	No	200 + page 2 data	PASS
TC-P06	Update own post	Yes (owner)	200 + updated doc	PASS
TC-P07	Update another user's post	Yes (other)	403 Forbidden	PASS
TC-P08	Delete own post	Yes (owner)	200 Deleted	PASS
TC-P09	Like a post	Yes	200 + updatedLikes[]	PASS
TC-P10	Unlike a post (toggle)	Yes	200 + likesDecrement	PASS

Places & Emergency Modules — Test Cases

TC-ID	Module	Test Description	Expected	Result
TC-PL01	Places	Filter by city + category	200 + filtered	PASS
TC-PL02	Places	Sort by rating DESC	200 + sorted list	PASS
TC-PL03	Places	Pagination limit=5 page=1	200 + 5 results	PASS

TC-ID	Module	Test Description	Expected	Result
TC-TP01	Tips	Search tips by city + keyword	200 + matched tips	PASS
TC-TP02	Tips	Filter by category (Safety)	200 + category results	PASS
TC-CM01	Companion	Create travel plan	201 + plan doc	PASS
TC-CM02	Companion	Send connection request	201 + request doc	PASS
TC-CM03	Companion	Accept connection request	200 + status:accepted	PASS
TC-EM01	Emergency	Log SOS with valid GPS coords	201 + log doc	PASS
TC-EM02	Emergency	Log SOS missing userId	400 Bad Request	PASS
TC-BL01	Safety	Block a user	200 + block record	PASS
TC-RP01	Safety	Report content with reason	201 + report doc	PASS

Test Coverage Summary

Module	Total TCs	Passed	Failed	Coverage
Authentication	8	8	0	100%
Posts	10	10	0	100%
Places	3	3	0	100%
Tips	2	2	0	100%
Companion	3	3	0	100%
Emergency	2	2	0	100%
Safety (Block/Report)	2	2	0	100%
TOTAL	**30**	**30**	**0**	

08 Conclusion

SoloSphere demonstrates the practical application of enterprise-grade software engineering principles in a domain-relevant problem context. The architecture choices — Three-Layer MVC, Dependency Injection, Strategy Pattern, Repository Pattern, and stateless JWT authentication — are not academic exercises but solutions to real scalability and maintainability challenges.

All five SOLID principles are observable at the code level, not merely aspirational. TypeScript enforces Single Responsibility through type-narrowed interfaces; the open/closed strategy maps allow feature extension without modification; Dependency Inversion makes every layer testable in isolation. The 30 test cases, all passing, confirm that the implementation matches the specification.

The project is deployed live on Vercel, serving real HTTPS traffic, which validates the deployment architecture decisions including serverless Express, MongoDB Atlas free tier, and CORS-aware API surface. The documented scalability roadmap (Redis caching, microservices, WebSocket SOS) provides a

credible path to production-grade scale.

Metric	Value
Total Source Files	~40 TypeScript + JSX files
Backend Endpoints	25+ REST endpoints across 6 modules
Database Collections	9 Mongoose-modelled collections
Design Patterns Implemented	6 primary + 12 supporting patterns
SOLID Principles Coverage	All 5 (S-O-L-I-D) with code evidence
Test Cases (All Passing)	30 / 30 (100%)
Deployment Platform	Vercel — Frontend + Serverless API
Live URL	solosphere-fs.vercel.app